# Sprint 3: Rubric Self-Assessment

**Chris Lee, Jaimin Fee, Kurt Schimmel (Team 6)**

*Review of our Sprint 3 deliverables against the grading rubric (internal, not for submission)*

## How to read this

This review grades our two Sprint 3 deliverables against the Sprint 3 rubric (100 points, 3% of the course grade):

- **Main doc:** *WhosDown (Team 6) Sprint 3* (.pdf, 33 pages)
- **Report:** *WhosDown (Team 6) Sprint 3 - User Interviews Report* (.pdf, 3-page body + an appendix of interview notes)
- **Wireframes:** *WhosDown (Team 6) Sprint 3 - Mapped Wireframes* (.pdf, 32 landscape pages: all 63 marked-up screens, two per page)
- **Playground:** `WhosDownWeather.playground`

These four items are the whole submission. Every screenshot is inside the main doc. The Figma file and the Firebase project `whosdown-57e99` are live extras the TA can open.

The deliverables have no printed page numbers, so "p." below means the page number in the PDF viewer. "§" is the section number inside the document.

Scores are our own estimate of what a grader following the rubric would give. Where a criterion depends on something outside the documents (TA approval, file access), the estimate says so.

## Score summary

| Criterion | Max | Estimate | Main evidence |
|---|---|---|---|
| Creation of Interactive Prototype | 15 | **15** | Main doc §6 (p. 23); Figma prototype, 63 hi-fi screens, all reachable |
| User Testing of Prototype | 10 | **10** | Report §1–6 (pp. 1–3), Appendix (pp. 4–6) |
| Identify Data Needs; Mapping | 30 | **30** | Main doc §8 (pp. 24–29); Mapped Wireframes PDF (all 63 screens) |
| Data Sources – External API data parsed | 8 | **8** | Main doc §4 (pp. 17–22); `WhosDownWeather.playground` |
| Data Sources – Internal API created | 7 | **7** (not applicable) | Main doc §1 (p. 1), §5 (p. 22) |
| Data Sources – Firebase structure established | 8 | **8** | Main doc §2 (pp. 1–11); console screenshots (pp. 7–8); TA Viewer access |
| Data Sources – Internal data store established | 7 | **7** (not applicable) | Main doc §5 (p. 22) |
| Feature Delivery Schedule | 15 | **15 if the TA approves the list; 11 if not** | Main doc §9 (pp. 29–33) |
| **Total** | **100** | **100** (96 without TA approval) | |

## 1. Creation of Interactive Prototype — 15 / 15

**What the rubric asks for (15 points):** all screens mocked up, and the prototype mimics

essential functionality.

**What we delivered**

- A clickable Figma prototype built from **63 high-fidelity screens (H01–H63)**, linked from Main doc §6 (p. 23).
- §6 lists the **12 flows** it covers: sign-up and onboarding, sign-in and guest, finding a game, joining with approval, full game and waitlist, coordination (chat, leave), hosting, managing a game (approve, edit, transfer, cancel), social (friends, messages, groups), history and reputation (past games, reviews), notifications, and profile and settings.
- We checked the prototype's links in Figma: **all 63 screens can be reached from the start screen (H01)**. The last three that were not linked (Guest Join Gate H34, Declined H35, Post-game Review H30) were connected on Oct 25. Guest mode uses a prototype variable, so "Request to join" opens the guest gate only for guests.
- The tab bar works from every main screen, so a tester can move around freely instead of following one fixed path.

**Why full points:** every screen in our wireframe set exists in high fidelity and is reachable, and every core action in the app (find, evaluate, join, host, manage, chat, review, notifications, settings) can be clicked through. That is the rubric's description of "complete".

**Small risks**

- The Figma file is still named "Who'sDown — Lo-fi Wireframes". The prototype itself starts on the hi-fi page, but renaming the file would avoid confusion.
- A few outcomes are simulated (for example, tapping the Pending badge shows the Declined screen). This is normal for a prototype that saves no data.

# 2. User Testing of Prototype — 10 / 10

**What the rubric asks for (10 points):** the prototype was tested with users, and key lessons (positive and/or negative) were identified. Partial credit (6) is for testing with only one or two users and few lessons.

**What we delivered**

- **Four participants** (P1–P4) tested the same Sprint 3 prototype: Report §1 Method (p. 1), with each participant's focus listed.
- **What worked:** 12 positive findings with the participants who said them, for example the clean interface, the map, how easy it is to post a game, prices shown before joining, and the roster (Report §2, p. 1).
- **23 lessons learned**, each with who raised it, what we will change, and the release it goes into (Report §3, pp. 1–3).
- **The lessons changed the plan:** three features were added to Version 1 (reminders 1.24, reporting 1.25, layout pass 1.26), nine Version 1 features were clarified, and six Version 2 features were added (Report §5, p. 3; Main doc §9.1, p. 29).
- Full interview notes for all four participants, including screen-by-screen positives and negatives for P3 and P4 (Report Appendix, pp. 4–6).
- Limitations stated honestly: small sample, and a prototype that cannot save data (Report §6, p. 3).
- The report body fits the spec's 2–3 page limit (3 pages); the interview notes are in the appendix.

**Why full points:** the rubric's partial-credit level is "one or two users" and "few lessons". We tested with four, and identified 23 lessons with both positive and negative findings, each tied to a concrete change. The table and bullet format matches the spec's recommendation.

**Small risks**

- Much of the feedback is feature requests rather than measured task results (no success rates or times). The rubric does not require measurements, but a short note on how sessions were run (in person, how long, which tasks) would make the method stronger.

# 3. Identify Data Needs; Mapping — 30 / 30

**What the rubric asks for (30 points):** all endpoints (or other data sources) the app needs have been identified, and the high-fidelity wireframes show where each piece of data is displayed or entered by the user.

**What we delivered**

- **Marked-up hi-fi wireframes:** the Mapped Wireframes PDF holds all **63 hi-fi screens**, two per page, each next to its source panel; the color key and endpoint IDs are in Main doc §8.1–8.2. Main doc §8.3 (p. 29) points to it. Every piece of data has a numbered badge, color-coded by source (Firestore, Auth, Storage, Open-Meteo, MapKit, user input, computed), and the panel lists every badge's source and field, the endpoints the screen uses, and its release. The same markup is live on Figma page "05 — Sprint 3 Data Mapping" (link in §8.3, p. 29).
- §8 opens with H09 Game Detail as a worked example: the marked-up screen and a table of its 16 badges, endpoints, and release (p. 25).
- **Every data source identified:** because we use Firebase, our "endpoints" are the Firestore reads and writes the app makes. §8.2 lists **19 reads (R1–R19)** and **19 writes (W1–W19)**, each with its exact path or query and the screens that use it, plus Auth, Storage, FCM, Open-Meteo, MapKit, and EventKit calls (pp. 26–29).
- The panels name the Firestore fields and struct properties from §2–3 (pp. 2–17), so every field shown has a matching property.

**Why full points:** the rubric's top level needs two things, *all* sources identified and the wireframes showing where data appears. We have both: the full endpoint list (R1–R19, W1–W19, and other sources) and visual markup on every one of the 63 screens, in the submitted Mapped Wireframes PDF. Nothing on a screen is left unmapped.

**Small risks:** the PDF pages are images, so the TA zooms in to read the badges (the endpoint list in §8.2 is searchable text). The PDF must be included in the submission; without it a grader sees only the H09 example.

# 4. Data Sources – External API data parsed — 8 / 8

**What the rubric asks for (8 points):** the team shows it can parse an external API's output, provides the structs that store the data for the interface, and provides one or more playgrounds that retrieve and parse it.

**What we delivered**

- **Why we use it:** Open-Meteo gives weather for outdoor games, is free, and needs no key (Main doc §4.1, p. 17).
- **Request and response** shown exactly (§4.2–4.3, pp. 17–18).
- **Structs:** `ForecastResponse`, `HourlyUnits`, `Hourly`, and the app-side `HourlyForecast`, with `CodingKeys` for the snake_case fields (§4.4, pp. 18–20).
- **Playground (submitted):** `WhosDownWeather.playground` requests the forecast with async/await, decodes it, and picks the hour closest to each game's start. Its printed output is in §4.5 (p. 20), and a screenshot of it running in Xcode with the console output is on p. 21.
- The same structs also run in our iOS test app, using games decoded from Firestore data (screenshot, p. 22).

- MapKit is covered too, as an on-device framework that needs no playground (§4.6, p. 22).

**Why full points:** all three parts of the rubric (parsing shown, app-side structs, a working playground) are present, and the playground has been run, not just written: the Xcode screenshot shows its real output.

# 5. Data Sources – Internal API created — 7 / 7 (not applicable)

**What the rubric asks for (7 points, "if applicable"):** an internal API on a cloud service with working endpoints.

**What we delivered:** Main doc §1 (p. 1) explains why we do not build our own API: all app data is created by our users, and Firebase provides authentication, a real-time database, storage, and push messaging without a server. §5 (p. 22) states plainly that the internal API is **not applicable**, and §8.2 lists every Firebase call the app makes instead.

**Why full points:** the spec says teams using Firebase document their Firebase structure instead, which we do in §2.

**Small risk:** unlike two other criteria, this rubric line does not literally say "if not applicable, full credit given". Confirm with the TA that Firebase teams get these points (this is the usual practice, and the spec's own wording supports it).

# 6. Data Sources – Firebase structure established — 8 / 8

**What the rubric asks for (8 points):** a JSON representation of **all** collections, each with at least one record populated, in document form; plus screenshots of the populated collections in Firebase, or TA access to the account.

**What we delivered**

- **Collection tree** of all 7 top-level collections and 6 subcollections, with design notes (Main doc §2.1, p. 1).
- **A populated JSON example for every collection and subcollection:** users, private settings, friends, notifications, venues, games, RSVPs, groups, members, conversations, messages, reviews, and reports (§2.2, pp. 2–6).
- **Proof the database exists and is populated** (§2.3, pp. 6–11):
  - 23 sample documents written to our real Firebase project `whosdown-57e99`.
  - Four Firebase console screenshots: the `users` collection, all 7 top-level collections with a game open, and the `rsvps` and `messages` subcollections (pp. 7–8).
  - Our test app decoding all 23 documents into the Swift structs (23 / 23, p. 9) and reading them back from Firestore ("Read 23 documents from 13 collections", pp. 10–11).
- **TA access:** our supervising TA has Viewer access to the Firebase project (stated on p. 6).

**Why full points:** the rubric accepts *either* console screenshots *or* TA access. We provide both, plus decoding proof that the JSON and the structs match.

**Small risk:** Firestore is in test mode, which stops allowing writes on Nov 24, 2026. Reading the data in the console is not affected, so grading is not at risk.

# 7. Data Sources – Internal data store established — 7 / 7 (not applicable)

**What the rubric asks for (7 points, "if applicable"; "if not applicable, full credit given"):** SwiftData model classes for on-device storage.

**What we delivered:** Main doc §5 (p. 22) states that WhosDown does not use SwiftData. It explains that Firestore's offline cache keeps recent games, RSVPs, and messages available without a connection, and that small view preferences use `@AppStorage`.

**Why full points:** the rubric explicitly gives full credit when this does not apply, and we explain why it does not.

# 8. Feature Delivery Schedule — 15 / 15 if the TA approves the list (11 otherwise)

**What the rubric asks for (15 points):** a feasible delivery schedule for every app feature; clear which features are in Version 1 and which in Version 2; **the list approved by the TA in advance**. The 11-point level is "mostly complete … some gaps", and also covers features in the prototype that are missing from the lists.

**What we delivered**

- **Scope rules** defining Version 1 (end of Sprint 4, core loop working end to end with real Firestore data) and Version 2 (final release), and what changed because of user testing (Main doc §9.1, p. 29).
- **26 Version 1 features (1.1–1.26)**, each with what works, the screens it covers, and the week it is built (§9.2, pp. 29–31).
- **23 Version 2 features (2.1–2.23)** in the same format (§9.3, pp. 31–32).
- **Not planned** items, so nothing in the prototype is left unexplained (§9.4, p. 32).
- **Risks to the schedule** and how we will handle them (§9.5, p. 33).
- Every feature lists its screens, so it is easy to check that all 63 prototype screens are covered.

**Why 15 is reachable:** the schedule is complete (every prototype feature is placed in V1, V2, or "not planned"), feasible (V1 is the core loop; push notifications and anything needing a server move to V2), and week-by-week.

**What keeps it from 15 today:** the rubric requires **TA approval in advance**. Until the TA approves the list, a grader could stop at the 11-point level. **Send the feature list (Main doc §9, or the separate feature list PDF in `Sprint3_Version2/`) to the TA for approval before the deadline, and submit it on Canvas as the spec requires.**

# Before you submit

| # | Action | Protects |
|---|---|---|
| 1 | Get the TA's approval of the feature list, and submit the list on Canvas | 4 points (Feature Delivery Schedule) |
| 2 | Confirm with the TA that the "Internal API" points are given to Firebase teams | 7 points |
| 3 | Rename the Figma file from "Lo-fi Wireframes" to a hi-fi name and share it with view access (optional; the markup is already in the document) | Prototype (15), clarity only |

| # | Action | Protects |
|---|---|---|
| 4 | Submit `WhosDown (Team 6) Sprint 3 Submission.zip`: the main doc, interviews report, and Mapped Wireframes as PDFs, plus `WhosDownWeather.playground` | All criteria |
| 5 | Send to Prof. H and the supervising TA by the Canvas deadline | All criteria |